Running the eDNAVisualJointModel re-runs on stats3

EJ45

2026-10-13

Quick orientation

These are the re-runs that the move to the 35°N study domain needed (ROADMAP_2026-10-06, 6.0; the remaining ones are in ROADMAP_2026-10-21, section 3). As with the FinWhale pipeline, the cleanest set-up is **SSH + tmux**: every job is a non-interactive `Rscript` that writes files.

> **Priority before the October 2026 presentation.** Run stage 1, then the joint comparison (stage 3) for seeds 2 and 3, then a second wave for seeds 4 and 5. That gives eDNA-only, visual-only and joint fits on several simulations of the real study domain. Stages 2 and 4 can wait. The visual validation and the explainer's visual fits are already done (run on the Mac on 2026-10-12). Seed 1 needs the data-based starting field (from 2026-10-13; `git pull` first), because from a flat start its eDNA fits get stuck at its extreme hake hot spots (ROADMAP_2026-10-06, 6.0). Run it as its own wave: `CMP_SEEDS="1" tools/stats3_reruns.sh stage 3`.

stats3 specs (from the IT FAQ): Intel i9-13900K with 24 physical cores and 32 hyperthreads, 128 GB RAM, RAID-1 NVMe (`/fast_raid`) and HDD (`/local_raid`).

Each Stan chain uses one core, and a fit runs 2–4 chains. No fit uses within-chain threading, so a single fit never uses more than 4 cores. Memory is not a constraint (the largest arrays are tens of MB per fit), so **cores are the budget: keep the total at or below 24.** Hyperthreads add little for Stan.

One script, `tools/stats3_reruns.sh`, runs everything. It groups the re-runs into five **stages**, with one tmux window and one log (`logs/<job>.log`) per job. Every job runs under `nice` and stops at its first failing step.

Stage	Jobs	Cores	Rough time
1	`prep`: real data (`01_load_muri.r`, `02_check_muri.r`), smoke test, `tests/visual_detection.R`, compile every model	2	about 1 h
2	HSGP4eDNA 2-D validation (4 jobs), visual validation (normal and sea state), joint simulator check	23	hours; `hsgp_bathysp_depthspref` 1–2 days
3	Joint comparison: modes `visual`, `edna`, `joint`, `joint_conv` (4 chains each) for every seed in `CMP_SEEDS` (default `"2 3"`)	16 per seed	about 4–6 h each (`visual` about 30 min)

Stage	Jobs	Cores	Rough time
4	Explainer fits: simulated eDNA and visual, real visual (budget 800 and 1600, both species)	14	about 1–3 h each
5	`finish`: comparison tables and `joint_compare_s<seed>.png` for every seed with fits, the remaining explainer figures	1	minutes

The real eDNA explainer fit (`expl_edna_real`) is in no stage: it waits for the data in ROADMAP_2026-10-21, section 5 (Ct values, volumes, dilutions), and is launched on its own with `tools/stats3_reruns.sh launch expl_edna_real` once they are in.

The times are estimates. The budget is now 800 functions and the domain is larger, which roughly doubles the old run times. `bathysp_depthspref` (1185 bottles) took 15 h on the old set-up. The whole list is about 280 core-hours, or about 2 days of wall time on stats3.

Stage 1 must finish first, because it checks the real data and compiles every Stan model. Stages 2 to 4 each need only stage 1, and stage 5 needs 3 and 4.

Part 1: Connect to stats3 (every session)

This is the same as for FinWhale. From a Mac terminal:

```
ssh -J ej45@jump.st-andrews.ac.uk ej45@maths-stats3.st-andrews.ac.uk
```

With the `~/.ssh/config` block from the FinWhale guide, this is just `ssh stats3`. Work on the fast disk:

```
cd /fast_raid/ej45
```

Part 2: SSH keys (optional, one time)

If you already set up keys and the `jump` / `stats3` hosts for FinWhale, skip this. Otherwise, follow Part 2 of the FinWhale guide:

- `ssh-keygen`;
- `ssh-copy-id` to both hops;
- the `Host jump` / `Host stats3` block in `~/.ssh/config`.

The `scp` and `rsync` commands below use the `stats3` alias. Without it, give the full host name and the jump host:

```
scp -J ej45@jump.st-andrews.ac.uk FILES \
    ej45@maths-stats3.st-andrews.ac.uk:/fast_raid/ej45/...
rsync -av -e 'ssh -J ej45@jump.st-andrews.ac.uk' \
    ej45@maths-stats3.st-andrews.ac.uk:/fast_raid/ej45/... DEST/
```

Part 3: Clone the repo and its submodule (one time)

On stats3:

```
cd /fast_raid/ej45
git clone --recurse-submodules https://github.com/MMARINeDNA/eDNAVisualJointModel.git
cd eDNAVisualJointModel
git submodule status        # external/HSGP4eDNA should show (v1.4)
```

eDNAVisualJointModel is public, but its submodule **HSGP4eDNA is private**. So the clone asks for a GitHub username and a **personal access token** (not your password). Create one at github.com → Settings → Developer settings → Personal access tokens:

- a fine-grained token: set *Resource owner* to `MMARINeDNA` and give it read access to `Contents`;
- or a classic token with the `repo` scope.

To avoid pasting the token on every pull, keep it in memory for a day:

```
git config --global credential.helper 'cache --timeout=86400'
```

To update later: `git pull && git submodule update --init`.

Part 4: Copy the private input data (one time, about 32 MB)

The raw MURI files are **not in the repo**, because the repo is public, and `.gitignore` keeps them out. Only the real-data loader (`R/01_load_muri.r`) and its check (`R/02_check_muri.r`) read them. Everything else uses the tracked, derived `data/processed/muri_{edna,visual}.rds`.

From a **Mac terminal in your main checkout**, which already has both files:

```
cd /Users/ej45/Documents/MMARINeDNA/eDNAVisualJointModel
scp data/MV1_MURI_df.csv data/SITEINFO_CCE_2018_Pp_Lo_Mn.csv \
    stats3:/fast_raid/ej45/eDNAVisualJointModel/data/
```

Then check on stats3:

```
ls -lh /fast_raid/ej45/eDNAVisualJointModel/data/
```

You should see `MV1_MURI_df.csv` at about 30 MB and `SITEINFO_CCE_2018_Pp_Lo_Mn.csv` at about 50 KB, next to the tracked `effort.csv`, `sightings.csv` and `hake_qPCR_MURI_df.csv`. If either private file is missing, the `prep` job stops with a message pointing here.

Bathymetry. On its first run, the loader downloads ETOPO 2022 bathymetry from NOAA (via `marmap`, a few MB) and caches it as `data/processed/etopo2022_60s_cce.rds`. If stats3 cannot reach NOAA, copy the cache across from a checkout that has one. The `stan-data-formatting` worktree does:

```
cd /Users/ej45/Documents/MMARINeDNA/eDNAVisualJointModel/.claude/worktrees
scp stan-data-formatting-61936f/data/processed/etopo2022_60s_cce.rds \
    stats3:/fast_raid/ej45/eDNAVisualJointModel/data/processed/
```

Part 5: R packages and CmdStan (one time)

Check R with `R --version`. The Mac runs R 4.5.2; 4.3 or later should be fine. Then:

```
cd /fast_raid/ej45/eDNAVisualJointModel
R
```

In R:

```
install.packages(c(
  "tidyverse", "posterior", "patchwork", "viridis", "bayesplot", "loo",
  "gridExtra", "data.table", "jsonlite", "sf", "terra", "marmap"
))
install.packages("cmdstanr",
  repos = c("https://stan-dev.r-universe.dev", getOption("repos")))

cmdstanr::check_cmdstan_toolchain(fix = TRUE)
cmdstanr::install_cmdstan(dir = "/fast_raid/ej45/cmdstan",
                          version = "2.38.0", cores = 8)   # same CmdStan as the Mac
cmdstanr::cmdstan_path()        # copy this path
quit(save = "no")
```

Packages install into `/local_raid/ej45/R/library` through the `.Rprofile` IT set up. `sf` and `terra` build against GDAL, GEOS and PROJ; if they fail with a missing-library error, ask maths-tech-support. The CmdStan build takes 5–10 minutes.

Point cmdstanr at CmdStan for every session (use the path printed above):

```
echo 'CMDSTAN=/fast_raid/ej45/cmdstan/cmdstan-2.38.0' >> ~/.Renviron
Rscript -e 'cmdstanr::cmdstan_version()'    # should print 2.38.0
```

Part 6: Run the re-runs inside tmux

The script creates the tmux session (`reruns`) itself. Start with the overview, then stage 1:

```
cd /fast_raid/ej45/eDNAVisualJointModel
tools/stats3_reruns.sh list
tools/stats3_reruns.sh stage 1
tools/stats3_reruns.sh status       # repeat until prep says DONE (about 1 h)
```

`prep` reruns the loader. The processed objects were already regenerated on the 35°N domain on `main` (#85), so the log should say each one **matches the committed copy**. If one differs, stop and look before going on. The smoke test's 26 checks must pass. If `prep` says `FAILED`, read `logs/prep.log`.

For the presentation, launch the joint comparison next. With the default `CMP_SEEDS="2 3"` it runs 8 jobs (32 chains for the first half hour, 24 once the two visual-only fits end), and should finish overnight:

```
tools/stats3_reruns.sh stage 3                      # seeds 2 and 3
CMP_SEEDS="4 5" tools/stats3_reruns.sh stage 3      # second wave, once the first is done
CMP_SEEDS="1" tools/stats3_reruns.sh stage 3        # seed 1 (needs the data-based start)
```

Sea-state comparison (ROADMAP_2026-10-21, section 3). Simulations with segment Beaufort, fitted visual-only, joint and joint with free conversion factors, each with the Beaufort covariate and without it (6 jobs, 24 cores per seed):

```
CMP_SEA=1 CMP_SEEDS="1" tools/stats3_reruns.sh stage 3
```

Results go to `outputs/joint/compare_sea_s<seed>/` and `compare_sea_nobft_s<seed>/`.

A single fit can be relaunched by name, e.g. `tools/stats3_reruns.sh launch cmp_s3_joint`. The tables and figure for a seed are rebuilt by `finish` (stage 5); for the presentation it is quicker to pull `outputs/joint/` back to the Mac (Part 9) and build them there.

The rest, after the presentation. Stage 2 is the full validation; it takes the longest:

```
tools/stats3_reruns.sh stage 2
```

It uses 23 cores. Its visual and joint-simulator jobs finish within hours, as do three of the four HSGP4eDNA jobs; `hsgp_bathysp_depthspref` (3 cores) runs for 1–2 days. Launch stage 4 when about 14 cores are free, and stage 5 once stages 3 and 4 are done:

```
tools/stats3_reruns.sh stage 4
tools/stats3_reruns.sh stage 5
```

Before launching a stage, check `uptime` (load average) or `htop`. Hold the stage back if other users are busy.

To watch a job: `tmux attach -t reruns`, then `Ctrl-b w` to pick a window. To detach, press `Ctrl-b d`; the jobs keep running after you log out. A finished job's window stays open at a shell prompt.

To re-run one job, e.g. after a failure:

```
tools/stats3_reruns.sh launch hsgp_bathygp_surface
```

What each job writes (git-ignored unless marked *tracked*):

Job	Output
`prep`	`data/processed/muri_{edna,visual}.rds` (*tracked*; should be unchanged), `outputs/smoke/`
`hsgp_*`	`external/HSGP4eDNA/outputs/<scenario>/` (fit summary, recovery CSV and PDF)
`visual_val`, `visual_val_sea`	`outputs/visual/validation/`, `outputs/visual/validation_seastate/`
`joint_sim`	`outputs/joint_sim/validation/`
`cmp_s<seed>_<mode>`	`outputs/joint/compare_s<seed>/fit_<mode>.rds`
`expl_*`	`outputs/explainer/`, `docs/figures/*.png` (*tracked*)
`finish`	`outputs/joint/compare_s<seed>/compare_*.csv`, `docs/figures/joint_compare_s<seed>.png` and the remaining explainer figures

Part 7: Check progress

From the Mac, without attaching:

```
ssh stats3 'cd /fast_raid/ej45/eDNAVisualJointModel && tools/stats3_reruns.sh status'
ssh stats3 'tail -n 30 /fast_raid/ej45/eDNAVisualJointModel/logs/visual_val.log'
```

Stan writes each chain's progress to the log (`Chain 2 Iteration: 400 / 1600`). The fit scripts print divergences, max Rhat and the run time when a fit ends, and warn when Rhat is above 1.05.

Part 8: If something goes wrong

- **A job says `FAILED`.** The end of its log has the R error. Fix the cause, then run `launch <job>` again. Other jobs carry on.
- **A model compile error in a stage 2–4 job** means `prep` did not finish. Run it first.
- **`tests/visual_detection.R` recompiles the visual model.** It runs inside `prep`, so don't relaunch `prep` while visual fits are running.
- **Stopping jobs.** To stop one, attach, pick its window and press `Ctrl-c`. To stop everything, run `tmux kill-session -t reruns`.

Part 9: Pull results back to the Mac

Leave the CmdStan chain CSVs on stats3. They can be about 330 MB per chain, and the fits keep what they need in `.rds` summaries. In your main checkout, on an up-to-date `main` with a clean tree:

```
cd /Users/ej45/Documents/MMARINeDNA/eDNAVisualJointModel
git switch main && git pull && git submodule update --init
R=stats3:/fast_raid/ej45/eDNAVisualJointModel
X="--exclude=*-202[0-9]*-[0-9]-*.csv"      # CmdStan chain CSVs

rsync -av "$X" $R/outputs/ outputs/
rsync -av "$X" $R/external/HSGP4eDNA/outputs/ external/HSGP4eDNA/outputs/
rsync -av $R/docs/figures/ docs/figures/
rsync -av $R/logs/ logs/
```

For the presentation only the joint comparison is needed, which is small (summaries only):

```
rsync -av "$X" $R/outputs/joint/ outputs/joint/
rsync -av $R/logs/ logs/
```

`docs/figures/*.png` then shows up as changed in `git status`. `data/processed/` is not copied back, because `prep` checked that it matches `main`.

Then ask Claude to do three things:

- rebuild the tables: the HSGP4eDNA README §4 (and a v1.5.x release), this README's results and ROADMAP_2026-10-21;
- rewrite the explainer's numbers, and re-render it locally with `quarto render docs/explainer.md --to html --embed-resources`;
- open the PRs.

TL;DR for repeat sessions

```
ssh stats3
cd /fast_raid/ej45/eDNAVisualJointModel
tools/stats3_reruns.sh status        # what is done, running or failed
tools/stats3_reruns.sh stage <n>     # launch the next stage
tmux attach -t reruns                # Ctrl-b w to pick a window, Ctrl-b d to detach
exit
```

Then the `rsync` block in Part 9 from the Mac.